

VISHNU

Design Decisions

Qosmic Runtime Audit Harness

Why I built it this way — architecture choices, options considered, and honest tradeoffs under a real timebox.

■ Time taken

4 hrs 15 min

■ Submission

Qosmic Take-Home · 2026

Companion files: [final_product_report.md](#) · [EVAL_LOOP.md](#) · [GitHub repo](#)

Contents

1. Introduction — what I was optimising for
2. High-level architecture
3. Phase model: single agent, strict order
4. Crawl design
5. Grounded checks as a write gate
6. Report format: Markdown canonical, HTML derived
7. Eval system design
8. Calibration integrity: Rule #1
9. Performance optimisations
10. What I deliberately did NOT build
11. Risks and known limitations
12. Summary table

1. Introduction

The assignment rewards a sharp self-improving eval loop on a thin runtime harness over infra novelty. Every decision here was made under a 4 hr 15 min timebox with that priority in mind. I'm writing this in first person because these were my tradeoffs.

What I was optimising for

Target	What it meant in practice
Audit output quality	Experiments cite real evidence; tech checks are grounded facts not guesses
Honest eval	10/10 structural ≠ content brilliance — documented explicitly
Generalization	No store-specific logic in skills/; zenrojas proves harness works unseen
Portability	AGENTS.md + skills/ works with Cursor, Claude Code, Codex, anything that reads markdown

Constraints that shaped every choice

Constraint	How it influenced design
~4h 15min timebox	Thin Python scripts not a framework; skills over custom runtime
Generalization required	No store-specific logic in skills/; cross-audit on zenrojas
Recruiter copy-paste check	Rule #1 — agent never reads target_report.md during audits
Eval weighted heavier	Two-layer eval (structural + optional judge) + self-improvement scripts
Any coding agent	AGENTS.md + skills/ portable across Cursor, Claude Code, Codex

2. High-level Architecture

The assignment's default is skills + an entry file. It also explicitly allows LangChain, Playwright, paid APIs, or a custom runtime. I chose the option that ships two full audits plus an eval system inside 4 hours 15 minutes.

Option	Description
A — Markdown harness + thin scripts ✓	Agent reads AGENTS.md + skills/; Python handles crawl, eval, HTML
B — LangChain / agent framework	Orchestrated chains, tool routing, memory
C — MCP servers	Custom tools exposed to agent via Model Context Protocol
D — Custom web app	UI + backend that runs audits
E — Full multi-agent orchestration	Subagents per phase with supervisor

Why not the alternatives

Alternative	Why skipped
LangChain / framework	Extra abstraction layer; harder to debug in take-home; assignment says not testing infra plumbing
MCP servers	Setup and maintenance overhead; Cursor built-in terminal + WebFetch is sufficient
Custom web app	Out of scope — deliverable is agent instructions + report files, not a product
Multi-agent orchestration	Merge conflicts on reports/{slug}.md; harder to verify end-to-end in one session

Advantages	Disadvantages
• Fast iteration — edit skills/*.md without touching Python	• No built-in agent memory across sessions — each audit starts fresh
• Portable — any agent that reads markdown can run this harness	• Phase discipline is soft — agent can skip steps unless post_audit.py catches failures
• Testable eval — eval.py runs in CI with no API keys required	• Auto-trigger via qosmic.mdc is Cursor-specific
• Clear separation — LLM does reasoning; Python does HTTP checks and HTML	

3. Phase Model: Single Agent, Strict Order

Qosmic audits go Crawl → Reason → Write. I added Grounded before write and Evaluate after write — honest tech checks need deterministic data, not LLM guesses.

Phase	File	Input	Output
1 — Crawl	skills/crawl.md	Store URL	artifacts/{slug}/ + manifest.json
2 — Reason	skills/reason.m d	artifacts/ + manifest	Draft: 10 experiments + competitors
3 — Grounded	eval/grounded. py	Store URL	tech_ground.json (required gate)
4 — Write	skills/write.md	Artifacts + grounded	reports/{slug}.md
5 — Evaluate	post_audit.py	reports/{slug}.md	eval/scores/{slug}.json + HTML

Advantages	Disadvantages
• Clear audit trail — each phase has explicit inputs and outputs on disk	• Longer wall-clock — full audit is 15–25 min end to end
• Easy to verify — check tech_ground.json timestamp vs report write time	• Soft enforcement — nothing stops agent writing before grounded except instructions
• Matches assignment phases — Crawl, Reason, Write are first-class	• Reason and grounded phases run sequentially, not in parallel

4. Crawl Design

4a. Representative 7-surface crawl — not full-site

The assignment asks for a representative set of surfaces. I needed bounded context the agent can read in one session without RAG or a vector store.

Surface key	Typical URL	Artifacts written
homepage	Store root	homepage.md + homepage_full.png
pdp_1 / pdp_2	Two /products/ URLs	pdp_N.md + pdp_N_full.png (×2)
collection	Collection / PLP	collection.md + collection_full.png
cart	{origin}/cart	cart.md + cart_full.png
faq_or_where_to_buy	FAQ, locator, /pages/...	faq_or_where_to_buy.md + png
content_page	Blog / content URL	content_page.md + content_page_full.png

Advantages	Disadvantages
• Bounded context — ~7 .md files fit agent context without RAG	• Misses edge pages — checkout, account, campaign LPs not in 7 surfaces
• Reproducible — manifest.json records URL, HTTP status, screenshot path per surface	• Heuristic URL picking — unusual store structures may get suboptimal PDPs
• Generalises — same script handles gingerpeople (404 cart) and zenrojas (200 cart)	• Targeted snapshot only — not site-wide link following

4b. requests for text · Playwright only for screenshots

crawl_lib.py uses requests for HTML text; screenshot_lib.py uses Playwright only for PNGs. Single browser session, domcontentloaded wait, fallback to load if PNG is blank (<5 KB).

Advantages	Disadvantages
• Fast text capture — HTTP fetch is seconds per page	• JS-rendered content may be incomplete for heavy SPAs
• Fewer bot blocks — plain HTTP often gets through where headless gets challenged	• Two code paths — text and screenshots can disagree if page changes between fetches
• Separation of concerns — text for reasoning, pixels for evidence citations	

5. Grounded Checks as a Write Gate

If the agent invents technical check values, recruiters catch fake Pass rows immediately — especially PSI scores. grounded.py runs before write and saves real values to artifacts/{slug}/tech_groundedException. The agent copies these verbatim into the report.

Check	Function	Notes
SSL Certificate	check_ssl()	Socket handshake + cert expiry
HTTPS Redirect	check_https_redirect()	HTTP → HTTPS 301/302
Robots.txt	check_url_exists()	GET /robots.txt → 200

Check	Function	Notes
Sitemap	check_url_exists()	GET /sitemap.xml → 200
Meta Tags & OG	parse_homepage_meta()	title, description, OG tags, favicon
PageSpeed Desktop	fetch_psi()	Google PSI API strategy=desktop
PageSpeed Mobile	fetch_psi()	Google PSI API strategy=mobile

Rule enforced: PSI 429 → Warn with 'Could not verify — rate limited'. Never fake a Pass. eval.py flags psi_mismatch_* if report PSI diverges from a fresh grounded run.

Advantages	Disadvantages
• Honest technical table — SSL and PSI are machine-verified, not hallucinated	• PSI rate limits without API key — parallel runs hit 429 faster
• Parallel PSI — mobile + desktop run concurrently via ThreadPoolExecutor (~50% faster)	• Extra step — skipping grounded produces dishonest reports; soft enforcement only
• Re-checkable — eval.py compares report values vs tech_groundeds.json on every run	

6. Report Format: Markdown Canonical, HTML Derived

reports/{slug}.md is the source of truth. render_report.py + templates/report.html produce polished HTML with zero extra LLM tokens. eval.py parses MD reliably with regex; CI runs structural checks on every reports/** change.

Advantages	Disadvantages
• Eval parses MD reliably — regex on experiment blocks, tables, executive summary	• Template maintenance — new sections need updates in both write.md and render_report.py
• Git-friendly diffs — markdown changes are reviewable in PRs	• Two artifacts — MD and HTML drift if someone edits MD without re-running post_audit.py
• Free HTML polish — KPI dashboard, pillar chart, tech-checks donut, embedded screenshots	
• Agent writes what it's good at — structured prose and tables, not CSS	

7. Eval System Design

The assignment weights Part 2 heavier than Part 1. This is where I spent the most design thinking. Three components: structural scoring (automatic), qualitative judge + challenger (optional), and cross-audit generalization check.

7a. Two-layer eval

Option	Description
Layer 1 — eval/eval.py ✓	Regex/parser, no API keys. Runs after every audit. Scores structural, field_quality, overall, flags[]

Option	Description
Layer 2 — Judge + challenger ✓	Optional LLM. Judge scores ≥ 7 triggers challenger to argue ≤ 5 . Gap $\geq 2 \rightarrow$ human_review: true in verdict JSON
LLM-only grading	Every report scored by LLM — cost + non-determinism problem
Human-only review	No automated scoring — does not scale beyond a handful of audits

Honesty note: 10/10 structural = passed the checklist (10 experiments, 5 pillars, field formats, table rows). It does not mean brilliant CRO experiments.

Advantages	Disadvantages
• CI-enforceable — eval.py runs on every reports/*.md change via GitHub Actions	• High structural score $\neq$ brilliant experiments — regex cannot judge insight
• Reproducible — same report $\rightarrow$ same structural score every time	• Judge step is manual — stale partial judge JSON removed before submission
• Separates failure modes — structure problems vs insight problems	• Confidence % in experiments is agent-written — format validated, not accuracy
• Cost control — default audit flow uses zero eval LLM tokens	

7b. Self-improvement loop

Nothing auto-edits skills/ or rubric.md. Scripts surface what needs changing; humans approve and commit. Git history of rubric.md IS the eval learning history.

Script	Trigger	Output	Purpose
pattern_tracker.py	Manual	recurring_patterns.json	Flags >50% stores $\rightarrow$ points to skill file to patch
calibrate_rubric.py	After editing outcomes/	calibration_report.json	Correlates lifted vs no-lift experiment qualities
run_full_eval.py	Manual	{slug}_verdict.json	Full loop + judge/challenger merge + human review flags

Truth hierarchy (most $\rightarrow$ least trusted): (1) eval/outcomes/ — market truth from real merchant data, (2) grounded checks — machine-verified facts, (3) judge + challenger consensus, (4) human spot-check — bootstrap only, shrinks over time.

Advantages	Disadvantages
• Honest about data — calibrate_rubric.py says LOW confidence until ≥ 20 shipped experiments	• Not autonomous yet — no cron; humans must run and approve changes
• Actionable flags — pattern_tracker maps recurring failures to specific skill files	• Bootstrap outcomes are sample data — not real merchant results yet
• Dual-LLM challenger — gap ≥ 2 flags human_review: true in verdict JSON	• Month 1–3 roadmap in EVAL_LOOP.md is a design plan, not live automation

8. Calibration Integrity: Rule #1

The assignment provides target_report.md as a quality bar for gingerpeople. Recruiters will check whether the gingerpeople audit copied it. Rule #1 — enforced in AGENTS.md, qosmic.mdc, and every skill file: the agent must NOT read target_report.md when auditing.

Advantages	Disadvantages
• Proves generalization — zenrojas.md is clearly different from gingerpeople and the target sample	• Harder first draft — agent may miss prose depth present in the human sample
• Recruiter-safe — no paraphrased experiments or reused exp-* IDs from the sample file	• More iteration needed — ~50 min on gingerpeople vs ~40 on zenrojas per AGENT_LOG
• Forces artifact-driven reasoning — every claim must cite crawl evidence, not the reference	• target_report.md sits in the repo; reviewers must be told clearly it is human-only

9. Performance Optimisations

Added after the first slow audits (~2+ min grounded, sequential screenshots). No new dependencies — all stdlib or existing Playwright.

Optimisation	File	Effect
Parallel PSI mobile + desktop	grounded.py ThreadPoolExecutor	~50% faster grounded phase
Parallel fast checks	grounded.py _run_parallel	Sub-second SSL, redirect, robots, sitemap, meta
Batch screenshots, single browser	screenshot_lib.py	One Chromium launch for all 7 surfaces
domcontentloaded + blank fallback	screenshot_lib.py	Faster than networkidle; retries if PNG < 5 KB
Cached tech_grounded.json	eval/eval.py read path	post_audit.py does not re-run PSI on every eval call

10. What I Deliberately Did NOT Build

Explicit decisions to skip — not oversights. Each has a documented path forward at scale.

Not built	Why skipped in 4h 15min	What I'd add at Qosmic scale
Vector DB / embeddings	~7 pages fits agent context; no RAG needed at this scale	Embed experiments for cross-merchant novelty detection at Month 3+
Firecrawl / paid crawl APIs	discover_store.py + sitemap sufficient for 7 surfaces	Add for stores with heavy bot protection or deep catalog discovery
LangChain / MCP servers	Not testing infra plumbing (assignment brief)	MCP tools if agents need standardised interfaces across IDEs
Autonomous cron	Manual triggers honest for bootstrap phase	Webhook after merchant ships experiment → auto-run calibrate_rubric.py

Not built	Why skipped in 4h 15min	What I'd add at Qosmic scale
Required LLM judge	Cost + non-determinism; structural eval sufficient for CI	Judge every report in production; challenger on all dimensions ≥ 7
Full-site link crawler	Noisy context, slow, not needed for 7-surface audit	Selective deep crawl for broken-link checks across full catalog

11. Risks and Known Limitations

Documented openly so reviewers see intentional tradeoffs, not blind spots.

Risk	What happens	Mitigation
Structural 10/10 $\neq$ content quality	eval.py checks format not CRO insight	Optional judge in rubric.md; human review for production
psi_mismatch_desktop flag	Report PSI at write time vs fresh grounded run differ	Flag in scores JSON; one-off, not recurring across stores
gingerpeople/cart 404	Real site behavior, not harness bug	Report cites artifacts/gingerpeople/cart.md status; experiment on cart recovery
PSI 429 without API key	Grounded returns Warn not Pass	.env.example provided + honest Warn rows in all reports
Bootstrap outcome data	eval/outcomes/ has realistic fake samples	calibrate_rubric.py is explicit about LOW confidence until real data arrives
Agent skips grounded	Soft enforcement via instructions only	Review tech_grounding.json timestamp vs report; re-run grounded and fix table

12. Summary Table

Every major decision in one place — what I chose, the main advantage, and the main tradeoff.

Decision	Chosen	Main advantage	Main tradeoff
Architecture	Markdown harness + thin Python	Portable, fast skill iteration	No cross-session agent memory
Orchestration	Single agent, 5 strict phases	Clear audit trail on disk	Long wall-clock; soft step enforcement
Crawl scope	7 fixed surfaces	Bounded context, no RAG	Misses edge pages
Text fetch	requests HTTP	Fast, fewer bot blocks	Incomplete JS-rendered pages possible
Screenshots	Required Playwright batch	Evidence + HTML embeds	~1–2 min extra; install friction
Tech checks	grounded.py before write	Honest Pass/Warn/Fail	PSI rate limits without API key
Report format	MD canonical, Python HTML	CI-parseable, free polish	Two files to keep in sync
Eval layer 1	Structural eval.py auto	Reproducible, CI-ready	Does not score CRO insight quality
Eval layer 2	Optional judge + challenger	Content depth when needed	Manual; extra LLM cost
Cross-audit	gingerpeople vs zenrojas	Matches assignment test plan exactly	N=2 is a smoke test, not statistical proof
Self-improvement	Manual triggers	Honest bootstrap — no faked automation	Not autonomous yet
Calibration	Rule #1 — no target_report.md	Proves real generalization	Harder first draft; more iteration
Vector DB	Not built	Right choice at this scale	Won't scale to thousands of audits

For the full file map and operator guide, see `final_product_report.md`. For the autonomy plan, see `EVAL_LOOP.md`. For day-to-day agent workflows, see `WORKFLOWS.md`.